# Fact checking con evidenza evidenziata — testa per token

## Indice


**Fact checking con evidenza evidenziata — testa per token** **3**
1. Che cosa si vuole ottenere . . . 3
2. Come funziona, passo per passo . . . 3
   2.1 Il forward non cambia . . . 3
   2.2 Che cosa entra nella testa . . . 3
   2.3 Che cosa fa la testa . . . 4
   2.4 Perché il preambolo . . . 4
   2.5 Che cosa si mostra all'utente . . . 4
   2.6 Che cosa si addestra . . . 4
3. Differenze con il piano per frase . . . 4
4. I modelli . . . 5
5. Glossario . . . 5
6. Sequenza di lavoro . . . 5
7. Obiettivi . . . 6

**Passo 1 — Modelli e hook per token** **6**
In breve . . . 6
1. Che cosa si cattura . . . 7
2. Hook sui vettori, per token . . . 7
3. La proiezione fissa (PCA) . . . 7
4. Spazio e memoria . . . 7
5. Test . . . 8

**Passo 2 — Dati dalle fonti** **8**
In breve . . . 8
1. Che cosa c'è già . . . 8
2. Il campo da aggiungere: `gold_char_sets` . . . 9
3. HoVer con pagine dal db . . . 9
4. Controlli . . . 9

**Passo 3 — Dati generati, etichette ricavate col modello, evidenza spostata** **10**
In breve . . . 10
1. Documenti generati con posizioni note (C2D, CG2C) . . . 10
2. Etichette ricavate col modello (ablazione) . . . 10
3. Evidenza spostata (righe sparse) . . . 10
4. Codice . . . 10

**Passo 4 — Split e mix** **11**
In breve . . . 11
1. Regole degli split . . . 11
2. I set . . . 11
3. Il mix `ev_tok_head` . . . 11
4. Comandi . . . 12

**Passo 5 — Il prompt con il preambolo** **12**

**Passo 6 — Rendering, etichette per token, estrazione dei vettori** **13**
In breve . . . 13
1. L'esempio renderizzato . . . 13
2. Dai caratteri del documento ai token . . . 13
3. L'etichetta di un token . . . 13
4. Documenti lunghi . . . 14
5. Che cosa si salva (fase 1a) . . . 14
6. Comando . . . 14
7. Test . . . 14

**Passo 7 — Metodi semplici da battere** **15**
In breve . . . 15
1. I metodi . . . 15
2. Comandi . . . 15
3. Tabella da compilare (una per modello) . . . 15

**Passo 8 — La testa, la loss, la decodifica** **16**
In breve . . . 16
1. La testa, passo per passo . . . 16
2. Il problema delle etichette . . . 17
3. La loss . . . 17
4. La decodifica: dai punteggi ai tratti . . . 18
5. Test . . . 19

**Passo 9 — Training** **19**
In breve . . . 19
1. Fase 1a . . . 20
2. Scegliere i parametri della decodifica . . . 20
3. Che cosa guardare durante il training . . . 20
4. Fase 1b: vettori completi, proiezione appresa . . . 21
5. Fase 2: training anche del modello . . . 21
6. Test prima di ogni nuova configurazione . . . 21

**Passo 10 — Valutazione** **21**
In breve . . . 21
1. Metriche sui caratteri (indipendenti da qualunque divisione in frasi) . . . 21
2. Metriche sulle frasi gold (quelle che contano) . . . 22
3. Confronto con la testa per frase . . . 22
4. Prove di comportamento . . . 22
5. Latenza . . . 22
6. Comandi . . . 23
7. Report finale (uno per modello) . . . 23

**Passo 11 — Esperimenti** **23**
In breve . . . 23
Regole . . . 23
La testa di partenza: H0 . . . 24
Esperimenti principali (in quest'ordine) . . . 24
Controlli sulla configurazione migliore . . . 24
Confronto con la testa per frase . . . 25
elephant-1 e gli altri modelli . . . 25
Controlli finali . . . 25
Registro . . . 25

**Passo 12 — Server e rilascio** **25**
In breve . . . 25
1. Che cosa fa il server per ogni richiesta . . . 26
2. Come gira ogni modello . . . 26

3. La cartella di rilascio . . . 26
4. Controlli prima del rilascio . . . 27
5. Congelamento e test . . . 27

**Checklist e problemi frequenti** **27**
1. Checklist per passo . . . 27
2. Se succede... . . . 28

**Una strada da esplorare: etichette per token dal modello stesso** **28**
Il problema . . . 28
L'idea . . . 28
Come farlo . . . 29
Quando tenerlo . . . 29
Rischi . . . 29
Perché vale la pena . . . 29

# Fact checking con evidenza evidenziata — testa per token

## 1. Che cosa si vuole ottenere

Come nel piano per frase (../testa_per_frase/00_overview.md): un decision model che, dato un documento e un claim, restituisce in **un solo forward** P(supportato) e, se il claim è supportato, **le parti del documento che lo dimostrano**.

La differenza è l'unità di uscita. Qui la testa dà un punteggio a **ogni token** del documento, non a ogni frase, e l'uscita sono **tratti di testo** (inizio e fine in caratteri):

```
{"p_supported": 0.93, "supported": true,
 "evidence": [{"start": 0,    "end": 61,   "score": 0.91},
              {"start": 4210, "end": 4262, "score": 0.84}]}
```

Perché: così il sistema **non dipende da come il testo è diviso in frasi**. Nessun divisore (NLTK o altro) serve al modello, né in training né in produzione. Funziona anche su testo senza punteggiatura, elenchi, tabelle in testo; se una frase lunga contiene l'evidenza solo in una sua parte, si può evidenziare solo quella.

Se poi si vogliono mostrare frasi intere all'utente, i tratti si "allargano" alla frase con qualunque divisore, solo in visualizzazione: la testa non ne dipende (§2.5).

## 2. Come funziona, passo per passo

### 2.1 Il forward non cambia

Il decision model fa **un solo forward su tutto il prompt** (preambolo con il claim, documento intero, claim, domanda), esattamente come nel piano binario. Il verdetto si legge dall'ultimo token, che ha visto tutto il documento. Non c'è nessun passaggio per frase o per token.

### 2.2 Che cosa entra nella testa

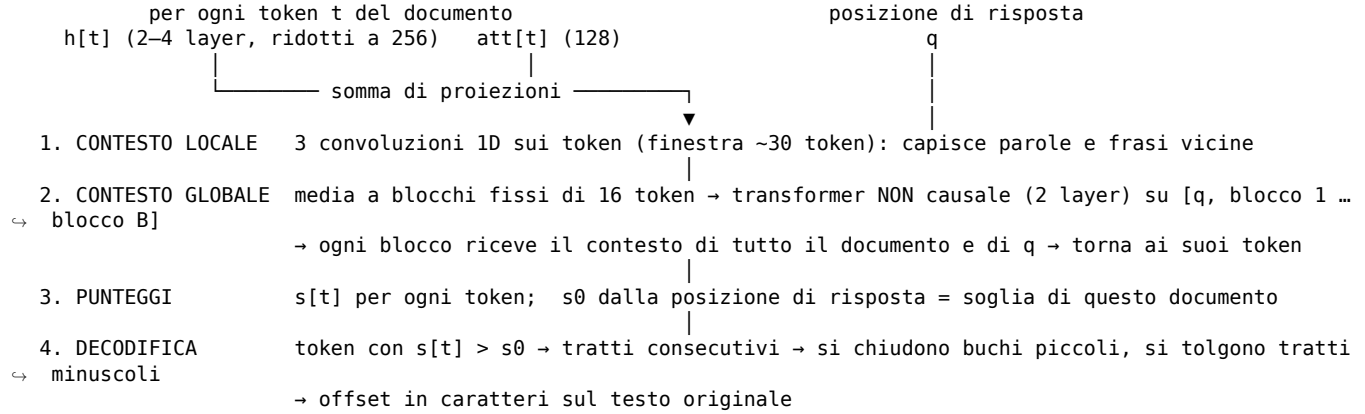
Dallo stesso forward, per **ogni token t del documento**:

| Nome | Che cos'è |
|---|---|
| `h[t]` | il vettore del token in alcuni layer (ridotto a 256 numeri, §2.3) |
| `att[t]` | quanto la posizione di risposta "guarda" quel token, negli 8 layer ad attenzione piena × 16 teste = 128 numeri |

e dalla posizione di risposta:

| Nome | Che cos'è |
|---|---|
| q | il vettore della posizione di risposta: ha visto tutto, sa il verdetto e quali parti servono |

### 2.3 Che cosa fa la testa




- **Contesto locale**: un token da solo non basta (una parola è evidenza perché sta in una certa frase); le convoluzioni guardano i token vicini.
- **Contesto globale**: l'evidenza sparsa (frase 1 e frase 40 insieme) richiede che parti lontane del documento si vedano. Un'attenzione piena su migliaia di token costa troppo; i **blocchi fissi di 16 token** (decisi solo dalla posizione, non dal significato) la rendono economica: un documento di 2.000 token diventa 125 blocchi. I blocchi sono interni alla testa: l'uscita resta per token.
- **Soglia del documento** `s0`: come nel piano per frase, si mostra ciò che supera la soglia di quel documento; nei casi multi-hop la soglia può scendere per far passare la parte "ponte".

### 2.4 Perché il preambolo

Identico al piano per frase (../testa_per_frase/05_formato_prompt.md): una riga `Claim under review: <claim>` prima del documento, così nei modelli causali (KnowLine) i token del documento sono letti sapendo cosa si verifica. Il resto del prompt è quello del piano binario.

### 2.5 Che cosa si mostra all'utente

La testa restituisce tratti di caratteri. Due modi di mostrarli, scelti dal client:

- **tratti esatti** (default);
- **frasi intere**: ogni tratto si allarga alla frase che lo contiene, con qualunque divisore. Il divisore sta nel client o nel server, fuori dal modello: cambiarlo non tocca la testa.

### 2.6 Che cosa si addestra

Solo la testa, a modello congelato: il verdetto non cambia. Le etichette sono le frasi di evidenza delle fonti, trasformate in intervalli di caratteri e poi in token (un token è "evidenza" se cade dentro una di quelle frasi). Le frasi delle fonti servono solo a creare le etichette: la testa impara token per token.

## 3. Differenze con il piano per frase

| | Testa per frase | Testa per token |
|---|---|---|
| Unità | frase (da un divisore) | token del modello |

| | Testa per frase | Testa per token |
|---|---|---|
| Divisore in produzione | sì, identico a quello del training | **no** |
| Uscita | indici di frase | tratti di caratteri |
| Elementi per documento | decine (mediana 46 su HoVer) | migliaia (mediana ~1.500 su HoVer) |
| Aggregatore | transformer su tutte le frasi | convoluzioni locali + transformer su blocchi fissi di token |
| Etichette | frasi gold sulla nostra divisione | frasi gold come intervalli di caratteri, poi token |
| Rumore delle etichette | basso | più alto: in una frase gold anche le parole irrilevanti sono "evidenza" (§04, loss in 08) |
| Salvataggio dei vettori | n frasi × d | n token × 256 (ridotti con una proiezione fissa, 06) |

## 4. I modelli

Gli stessi del piano per frase (../testa_per_frase/01_ambiente_e_modelli.md): i 10 decision model dello studio, tutti Qwen3.5 (32 layer, 8 ad attenzione piena ai layer 3, 7, …, 31). Si itera su **KnowLine-4B** (d = 2.560), si conferma su **elephant-1** (d = 4.096), poi gli altri.

## 5. Glossario

| Termine | Significato |
|---|---|
| token del documento | i token del prompt che appartengono al documento (per KnowLine, nelle richieste lette due volte, quelli della seconda copia) |
| tratto (span) | sequenza di token consecutivi segnati come evidenza, convertita in `[inizio, fine)` di caratteri |
| blocco | 16 token consecutivi del documento, unità interna della testa per il contesto globale |
| `s[t]`, `s0` | punteggio del token t; soglia del documento |
| frase gold | frase di evidenza indicata dalla fonte (SciFact, WiCE, HoVer), come intervallo di caratteri |
| colpita | una frase gold è "colpita" se almeno metà dei suoi caratteri è dentro i tratti mostrati |
| insieme completo | tutte le frasi gold di una riga colpite |
| sufficienza | il documento ridotto ai tratti mostrati fa ancora dire "supportato" al modello |

## 6. Sequenza di lavoro

| # | Passo | File | Prodotto |
|---|---|---|---|
| 1 | Modelli e hook per token | 01 | `evidence/models.py` con cattura per token |
| 2 | Dati dalle fonti (già costruiti) + intervalli gold di caratteri | 02 | `dataset/evidence/*.jsonl` con `gold_char_sets` |
| 3 | Dati generati | 03 | righe `construct`, `ablation`, sparse |

| # | Passo | File | Prodotto |
|---|---|---|---|
| 4 | Split e mix | 04 | set congelati, mix `ev_tok_head` |
| 5 | Prompt con preambolo | 05 | come nel piano per frase |
| 6 | Rendering, etichette per token, estrazione | 06 | vettori ridotti su disco |
| 7 | Baseline | 07 | numeri da battere |
| 8 | Testa, loss, decodifica | 08 | `evidence/token_head.py`, `token_losses.py`, `decode.py` |
| 9 | Training | 09 | teste addestrate |
| 10 | Valutazione | 10 | metriche su caratteri e su frasi |
| 11 | Esperimenti | 11 | configurazione finale |
| 12 | Server e rilascio | 12 | server senza divisore |
| — | Checklist | 13 | |
| — | Una strada da esplorare | 14 | |

### 7. Obiettivi

| Obiettivo | Soglia |
|---|---|
| Verdetto | uguale al piano binario (BAcc e AUC entro 0,3 punti) |
| F1 sui caratteri (tratti mostrati contro frasi gold) | ≥ E-LEX + 15 punti |
| Frasi gold colpite (recall) | ≥ 0,80 su HoVer |
| Insieme completo su HoVer | ≥ 0,65 |
| Evidenza lontana | F1 con frasi gold a ≥ 20 frasi di distanza al massimo 5 punti sotto quello con frasi vicine |
| Sufficienza | ≥ 85% |
| Tratti mostrati per riga | in media al massimo 5; nessun tratto sotto 4 token |
| Latenza | al massimo +3% del solo verdetto (modelli 4B: +3% o +3 ms) |

# Passo 1 — Modelli e hook per token

## In breve

Come nel piano per frase (../testa_per_frase/01_ambiente_e_modelli.md): `evidence/models.py` fa un forward con il codice degli autori di ciascun modello e, con degli hook, copia i dati che servono alla testa. La differenza: si conservano i vettori di **tutti i token del documento**, non un riassunto per frase. Perché stiano in memoria e su disco si riducono subito a 256 numeri per layer con una **proiezione fissa** (§3).

Tutto il resto è identico al piano per frase e non si ripete qui:

- pacchetti (`flash-linear-attention`, `causal-conv1d` per i layer DeltaNet);
- tabella dei modelli, codice degli autori per ciascuno, posizione di risposta;
- come trovare i layer (`text_layers`, `full_attention_layers`);
- la funzione `sdpa_capture` che ricalcola la riga di attenzione della posizione di risposta negli 8 layer pieni.

## 1. Che cosa si cattura

| Dato | Forma per riga | Da dove |
|---|---|---|
| `h` | n_token × L × 256 | uscita dei L layer scelti (partenza: 23 e 31; H6 prova 19, 23, 27, 31), solo nei token del documento, proiettata (§3) |
| `att` | n_token × 128 | riga di attenzione della posizione di risposta, negli 8 layer pieni × 16 teste, solo nei token del documento |
| `q` | L × d | stato della posizione di risposta (non proiettato: è uno solo) |
| `doc_tok` | [primo, ultimo] | intervallo dei token del documento nel prompt |
| `answer_logits` | 2 | per ricalcolare P(supportato) |

`att` per token viene gratis: la riga di attenzione è già calcolata su tutti i token; nel piano per frase la si sommava per frase, qui si tiene com'è.

## 2. Hook sui vettori, per token

```python
class TokenTap:
    """Per ogni layer scelto: i vettori dei token del documento, proiettati a 256, e lo stato della posizione di
    ↪ risposta."""
    def __init__(self, layers, layer_idx, projections):
        self.out, self.doc, self.answer_pos = {}, None, None        # doc = (primo, ultimo) token del documento
        self.proj = projections                                       # {layer: matrice fissa [D, 256]} (§3)
        self.handles = [layers[i].register_forward_hook(self._hook(i)) for i in layer_idx]

    def _hook(self, i):
        def hook(module, args, output):
            h = (output[0] if isinstance(output, tuple) else output)[0]          # [T, D]
            a, b = self.doc
            self.out[i] = {"tok": (h[a:b + 1].float() @ self.proj[i]).half(),   # [n, 256], subito sulla GPU
                           "q": h[self.answer_pos]}
        return hook
```

La proiezione si fa sulla GPU dentro l'hook: dalla GPU escono 256 numeri per token invece di 2.560 o 4.096.

## 3. La proiezione fissa (PCA)

Per salvare su disco i vettori di migliaia di token servono vettori corti. Si usa una PCA calcolata una volta per modello e per layer:

1. 2.000 righe a caso del mix di training, forward con un hook che raccoglie 200 token a caso per riga e layer (400.000 vettori per layer);
2. media e prime 256 componenti principali (`torch.pca_lowrank` sui vettori centrati);
3. si salvano in `data/evidence_tok/pca/<modello>/layer<i>.safetensors` (media [D], matrice [D, 256]) con la quota di varianza spiegata.

La proiezione fissa serve solo alla fase 1a (vettori salvati). Nella fase 1b, con il modello in memoria, la testa impara la propria proiezione dai vettori completi (09 §4); l'esperimento H9 dice se la PCA perde qualcosa.

## 4. Spazio e memoria

Per riga, con 2 layer, `att` in fp16, documento di n token:

| | Per token | HoVer (mediana ~1.500 token) | WiCE (95° percentile ~5.300 token) |
|---|---|---|---|
| `h` (2 × 256, fp16) | 1 KB | ~1,5 MB | ~5,3 MB |
| `att` (128, fp16) | 256 B | ~0,4 MB | ~1,4 MB |

Per le ~28.600 righe di SciFact, WiCE e HoVer: ~60 GB. Con 4 layer ~100 GB: per questo si parte da 2 layer.

### 5. Test

Quelli del piano per frase (verdetto invariato, layer giusti, attenzione uguale alla modalità “eager”, maschera conservata), più:

| Test | Deve dare |
|---|---|
| `doc_tok` decodificato è il documento (a meno dell’escaping del prompt) | identico, su 200 righe |
| PCA: varianza spiegata dalle 256 componenti | registrata; se sotto il 70% nel layer 31, provare 384 componenti |
| proiezione nell’hook = proiezione fatta dopo sugli stati completi | differenza < 1e-2 (fp16) |

## Passo 2 — Dati dalle fonti

### In breve

I dati di SciFact, WiCE e HoVer sono **già costruiti** dal codice in `evidence/data/` (../../../evidence/README.md): `dataset/evidence/<fonte>.jsonl`. Ogni riga ha documento, claim, etichetta, frasi di evidenza e il tipo di termine di loss.

Per la testa per token serve una cosa in più: le frasi di evidenza **come le dà la fonte**, in caratteri, indipendenti dal nostro divisore. Oggi il builder le usa per calcolare `evidence`, ma non le salva. Il lavoro di questo passo è salvarle.

### 1. Che cosa c’è già

| Fonte | Righe | Positivi | Note |
|---|---|---|---|
| SciFact | 1.258 | 508 | abstract corti; `contra` per i CONTRADICT |
| WiCE | 5.294 | 3.100 | documenti lunghi; 801 “parzialmente supportati” senza loss di evidenza; `meta.aggrefact_overlap` segna le pagine presenti in AggreFact |

| Fonte | Righe | Positivi | Note |
|---|---|---|---|
| HoVer | 22.063 | 12.974 | 2–4 hop con pagine di disturbo; `meta.gold_alignment` = `exact` (frasi HotpotQA) o `db_split` (pagina presa dal db di HoVer) |

Comando: `python -m evidence.data.build --source scifact wice hover --hover-db-fallback` (filtro di lunghezza `--max-tokens`, default 32.768 token di KnowLine-4B).

## 2. Il campo da aggiungere: `gold_char_sets`

```
"gold_char_sets": [[[412, 497], [3120, 3188]]],
"contra_char": []
```

- `gold_char_sets`: per ogni insieme alternativo di evidenza, la lista degli intervalli `[inizio, fine)` di caratteri delle frasi gold **della fonte** (le frasi di SciFact, gli elementi di WiCE, le frasi HotpotQA di HoVer);
- `contra_char`: lo stesso per le frasi che contraddicono (SciFact).

Modifica al codice (piccola):

1. in `evidence/data/schema.py`, `assemble(...)` salva `gold_char_sets` e `contra_char` (i `gold_sets` e `contra` che riceve già) accanto a `evidence`;
2. `validate(...)` controlla che ogni intervallo stia nel documento, non sia vuoto, e che `label = 0` abbia `gold_char_sets` vuoto;
3. un test in `tests/test_evidence_data.py`: il testo di ogni intervallo è la frase gold della fonte;
4. ricostruire i dataset.

`sentences` ed `evidence` (per frase) restano nella riga: servono alla valutazione a livello di frase (10 §3) e al confronto con il piano per frase. La testa per token non li usa.

## 3. HoVer con pagine dal db

Per le pagine prese dal db di HoVer (`gold_alignment = db_split`) la fonte non dà le frasi: l'intervallo gold è la nostra frase i di quella pagina, ottenuta con il nostro divisore. È l'unico punto in cui il divisore entra nei dati, e solo per creare etichette (mai in produzione). Misurato: la frase così ottenuta è quella giusta nel 98% dei casi (il 2% è una frase sfasata).

Regola per il mix e la valutazione (04):

- training: righe `db_split` tenute, con peso 0,5;
- dev e test: solo righe `exact`.

## 4. Controlli

| Controllo | Atteso |
|---|---|
| Ogni intervallo di `gold_char_sets` dentro il documento, non vuoto | tutte le righe |
| Testo degli intervalli = frase gold della fonte (a meno degli spazi ai bordi) | tutte le righe `exact` |
| 30 positivi per fonte letti a mano con gli intervalli evidenziati | le parti evidenziate dimostrano il claim, ≥ 28 su 30 |

# Passo 3 — Dati generati, etichette ricavate col modello, evidenza spostata

## In breve

Le stesse tre famiglie del piano per frase (../testa_per_frase/03_dati_generati.md), con una differenza: le etichette si salvano come **intervalli di caratteri** (`gold_char_sets`), così servono direttamente alla testa per token. Per le prime prove (SciFact, WiCE, HoVer) questo passo è facoltativo: serve per avere evidenza su testi non Wikipedia e più casi lontani.

## 1. Documenti generati con posizioni note (C2D, CG2C)

Procedura del piano per frase: il generatore scrive un articolo con le frasi date marcate `<e1>…</e1>`; si tolgono i tag e si annota dove stavano.

Per la testa per token si salva l'intervallo **esatto** del testo marcato, non la frase che lo contiene. È l'unica fonte in cui l'etichetta è precisa anche dentro una frase: se il generatore ha inserito il fatto in una frase più lunga, solo quella parte è evidenza. Queste righe sono il miglior segnale per insegnare tratti stretti (esperimento H10, 11).

Varianti come nel piano per frase: il negativo (si toglie un fatto necessario) e il claim parziale (evidenza = solo i fatti usati).

## 2. Etichette ricavate col modello (ablazione)

Per righe supportate senza evidenza annotata. L'ablazione toglie **frasi** dal documento e guarda il verdetto: per farlo serve dividere il documento in unità. Si usano le frasi del nostro divisore. È un lavoro offline sui dati di training, che non entra in produzione: la dipendenza dal divisore resta fuori dal modello.

Procedura identica al piano per frase (ricerca greedy del gruppo più piccolo di frasi che basta, con KnowLine-4B, giudice LLM, accordo con elephant-1). Le frasi trovate si salvano come intervalli di caratteri in `gold_char_sets`, con `evidence_quality = ablation` e peso 0,5.

## 3. Evidenza spostata (righe sparse)

Da righe con evidenza nota e almeno 2 frasi gold: le frasi gold si spostano agli estremi del documento, o si inseriscono frasi di disturbo fra di esse, o si aggiunge testo di disturbo all'inizio finché l'evidenza è oltre 1.024 o 4.096 token. Si spostano intere le frasi della fonte (non i token), e gli intervalli di `gold_char_sets` si ricalcolano sul documento nuovo con lo stesso `DocBuilder` del builder.

Controlli del piano per frase: frasi che iniziano riferendosi alla precedente ("He", "This", "The company") non si spostano da sole; il giudice conferma l'etichetta; le frasi di disturbo vengono solo da documenti di training.

## 4. Codice

| Script | Cosa fa |
|---|---|
| `evidence/data/gen_marked.py` | C2D e CG2C con intervalli esatti |
| `evidence/data/label_ablation.py` | ablazione con KnowLine-4B, uscita in intervalli di caratteri |
| `evidence/data/augment_sparse.py` | evidenza spostata |

Tutti scrivono righe nel formato di `evidence/data/schema.py` (con `gold_char_sets`) in `dataset/evidence/`, validate come le altre.

# Passo 4 — Split e mix

## In breve

1. **Split**: train / dev / test congelati, per documento (`doc_id`), con le fonti già costruite.
2. **Mix** `ev_tok_head`: le righe di training con il loro tipo di loss e i loro pesi.

Script: `evidence/data/make_splits.py`, `evidence/data/make_mix.py`. Uscita in `dataset/evidence/splits/` e `dataset/evidence/mixes/`.

## 1. Regole degli split

- Per `doc_id`: lo stesso documento sta in un solo split.
- **WiCE**: le righe con `meta.aggrefact_overlap = "dev"` o `"test"` (tutto il dev e il test di WiCE, più 2 righe del train) **non vanno in training**. Vanno in test (`test_evid_ood`), così restano utili come misura fuori dominio.
- **SciFact**: train d'origine → training e dev (90/10 per `doc_id`); validation d'origine → test.
- **HoVer**: train d'origine → training e dev (95/5 per `doc_id`); dev d'origine → test. Nei set di dev e test di HoVer si tengono solo le righe con `gold_alignment = exact`.
- Seme fisso (13); `manifest.json` con gli hash dei file e i conteggi.

## 2. I set

| Set | Contenuto | Uso |
|---|---|---|
| `train` | il resto | training della testa |
| `dev_evid_select` | positivi del dev (SciFact, HoVer) | scelte: configurazione, soglia di decodifica, esperimenti |
| `dev_evid_confirm` | metà del dev, separata per `doc_id` | conferma delle scelte |
| `dev_evid_neg` | negativi del dev con documenti lunghi | controllare che la testa non accenda tratti su claim non supportati |
| `test_evid_id` | test di HoVer (`exact`) e SciFact | misura finale |
| `test_evid_ood` | WiCE dev + test | misura finale fuori dominio (pagine web) |
| `test_evid_sparse` | da `test_evid_id`, con evidenza spostata lontano (passo 3) | evidenza lontana |

Per ogni set si salvano: frasi gold per riga, distanza fra la prima e l'ultima (in frasi e in token), lunghezza in token.

## 3. Il mix `ev_tok_head`

Ogni riga riceve dal suo `evidence_term` il tipo di loss (08 §3), più due pesi:

| Famiglia | Termine | Peso della riga | ω (peso dei token non gold) |
|---|---|---|---|
| HoVer supportati, `exact` | `supervised` | 1,0 | 0,5 |
| HoVer supportati, `db_split` | `supervised` | 0,5 | 0,5 |
| SciFact SUPPORT | `supervised` | 1,0 | 0,5 |
| WiCE train supportati | `supervised` | 1,0 | 0,3 |

| Famiglia | Termine | Peso della riga | ω (peso dei token non gold) |
|---|---|---|---|
| C2D con intervalli esatti (passo 3) | `supervised` | 1,0 | 1,0 |
| Etichettati col modello (passo 3) | `supervised` | 0,5 | 0,5 |
| Negativi (HoVer NOT_SUPPORTED, SciFact CONTRADICT/NEI, WiCE not_supported) | `mil_neg` | 1,0 | — |
| WiCE parzialmente supportati | `none` (escluse dal mix della testa) | — | — |

ω è più basso per WiCE perché i suoi gruppi di evidenza sono spesso incompleti (altre righe della pagina possono supportare il claim).

Vincoli che lo script controlla e stampa:

| Vincolo | Valore |
|---|---|
| Negativi | 20–30% delle righe |
| Positivi con 2 o più frasi gold | ≥ 40% |
| Positivi con evidenza oltre 1.024 token dall'inizio del documento | ≥ 15% |
| Documenti in comune con dev o test | 0 |
| Righe WiCE con `aggrefact_overlap` nel training | 0 |

### 4. Comandi

```
python -m evidence.data.make_splits --out dataset/evidence/splits
python -m evidence.data.make_mix --recipe recipes/ev_tok_head.json --out
↪  dataset/evidence/mixes/ev_tok_head.jsonl
```

## Passo 5 — Il prompt con il preambolo

Identico al piano per frase: ../testa_per_frase/05_formato_prompt.md. Riassunto:

- si aggiunge `Claim under review: <claim>` e una riga vuota **prima dello stato**, nello stesso turno; il resto del prompt è quello del piano binario, prodotto dal codice degli autori;
- si sceglie il testo del preambolo (P-A o P-B) misurando il verdetto su `dev_select` con e senza preambolo;
- se il verdetto del piano binario resta entro 0,3 punti di BAcc e AUC con il preambolo, lo si riusa; altrimenti lo si riaddestra con il preambolo;
- la scelta si scrive in `question.json` (`preamble`, `verdict`).

Due note specifiche della testa per token:

1. **KnowLine scrive due volte le richieste fino a 3.000 token.** I token del documento si prendono dalla **seconda** copia, che ha già visto il claim e l'intero documento della prima. Il preambolo fa parte dello stato e compare in entrambe le copie.
2. **Nei modelli bidirezionali** (elephant-1 e altri) i token del documento vedono il claim negli 8 layer pieni anche senza preambolo; l'esperimento H8 (11) misura quanto serve.

# Passo 6 — Rendering, etichette per token, estrazione dei vettori

## In breve

1. **Rendering** (`evidence/token_render.py`): per ogni riga si costruisce il prompt con il preambolo (codice degli autori) e si calcola, per **ogni token del documento**, a quali caratteri del documento originale corrisponde. Da qui l'etichetta del token (dentro o fuori da una frase gold) e, in produzione, la conversione dei tratti in caratteri.
2. **Estrazione** (`evidence/token_extract.py`): un forward per riga con gli hook del passo 1; si salvano su disco i vettori ridotti di ogni token, la sua attenzione, `q`. Il modello è congelato: si calcolano una volta.

## 1. L'esempio renderizzato


```
{"id": "hover:train:…", "model": "knowline-4b",
 "input_ids": [248045, 846, "…"], "answer_pos": 2371,
 "doc_tok": [1103, 2290],
 "tok_char": [[0, 7], [7, 11], [11, 19], "…"],
 "tok_label": [0, 0, 1, 1, 1, "…"],
 "gold_tok_spans": [[[37, 61], [612, 640]]],
 "label": 1, "evidence_term": "supervised", "row_weight": 1.0, "omega": 0.5}
```


| Campo | Significato |
|---|---|
| `doc_tok` | primo e ultimo token del documento nel prompt (KnowLine letto due volte: della seconda copia) |
| `tok_char[i]` | caratteri `[inizio, fine)` del documento **originale** coperti dal token `doc_tok[0] + i` |
| `tok_label[i]` | 1 se il token sta dentro una frase gold (§3) |
| `gold_tok_spans` | per ogni insieme alternativo, gli intervalli di token di ogni frase gold (servono alla loss per frase gold, 08 §3) |

## 2. Dai caratteri del documento ai token

Il documento nel prompt può essere scritto diversamente dall'originale: dentro un JSON (" → \", a capo → \n) per elephant-1, StartLux, d3-flash, Jeffan, Nimble, pplx e Clef; come testo indentato per KnowLine. La procedura del piano per frase (../testa_per_frase/06_rendering_ed_estrazione.md §2) va qui al contrario: serve, per ogni token, la sua posizione nel testo **originale**.

1. Costruire il prompt in tre pezzi (prima del documento, documento come appare nel prompt, dopo); per il pezzo del documento, `escape_with_map` dà per ogni carattere originale la sua posizione nel testo trasformato.
2. Invertire la mappa: per ogni posizione del testo trasformato, il carattere originale da cui viene (un \" di due caratteri torna a un solo ").
3. Tokenizzare il prompt con `return_offsets_mapping=True`.
4. Per ogni token nel pezzo del documento: `tok_char` = primo e ultimo carattere originale che copre. I token fatti solo di escape o spazi ereditano l'intervallo vuoto `[c, c]`.

`escape_with_map` è già scritta e verificata (identica a `json.dumps` su 20.000 stringhe difficili): si sposta in `evidence/escape.py` e si usa sia qui sia nel piano per frase.

## 3. L'etichetta di un token

```
tok_label[i] = 1  se almeno metà dei caratteri non spazio del token sta dentro un intervallo di gold_char_sets
             = 0  altrimenti (e sempre 0 se la riga ha label 0)
```

Esempio: la frase gold "Rose Byrne is an Australian actress." diventa ~8 token tutti con etichetta 1; i token di "Rose Byrne" come titolo della pagina, fuori dalla frase, restano 0.

`gold_tok_spans`: per ogni frase gold, il primo e l'ultimo token con etichetta 1 al suo interno.

Rumore da sapere: in una frase gold anche le parole irrilevanti hanno etichetta 1 (le fonti annotano frasi, non parole). La loss ne tiene conto (08 §3): chiede soprattutto che **ogni frase gold sia colpita** e che **fuori dalle frasi gold non si accenda nulla**, non che ogni token di una frase gold sia sopra soglia.

## 4. Documenti lunghi

Il filtro di lunghezza del build (02) ha già tolto le righe oltre il contesto. Per la fase 1b (modello in memoria) vale il budget di 8.192 token: le righe oltre si escludono da quella fase (oggi 75 righe di WiCE, nessuna di HoVer e SciFact).

## 5. Che cosa si salva (fase 1a)

| Feature | Forma | Tipo |
|---|---|---|
| `h` | n_token × L × 256 | fp16, proiettato con la PCA del passo 1 |
| `att` | n_token × 128 | fp16 |
| `q` | L × d | bf16 |
| `tok_char`, `tok_label` | n_token × 2, n_token | int32, int8 |
| `answer_logits` | 2 | fp32 |

Formato: `data/evidence_tok/features/<modello>/<set>/shard-00000.safetensors`, 500 righe per file (le righe hanno lunghezze molto diverse), più `index.jsonl` con `id`, shard, offset del primo token e n. `meta.json` con modello, pesi, preambolo, layer, file PCA, hash di `token_render.py`: il training rifiuta vettori calcolati con impostazioni diverse.

## 6. Comando

```
gpu-run --batch --name evtok-extract-knowline -- bash -c \
  'python -m evidence.token_extract --model knowline-4b \
     --rows dataset/evidence/mixes/ev_tok_head.jsonl dataset/evidence/splits/dev_evid_select.jsonl \
            dataset/evidence/splits/dev_evid_confirm.jsonl dataset/evidence/splits/dev_evid_neg.jsonl \
     --out data/evidence_tok/features/knowline-4b > logs/evtok-extract-knowline.out 2>&1'
```

Riprende dagli shard completi; ordina le righe per lunghezza; batch 1 sopra i 4.096 token.

## 7. Test

| Test | Deve dare |
|---|---|
| Testo ricostruito dai `tok_char` dei token del documento = documento originale (a meno degli spazi) | 200 righe su 200, per ogni modello |
| Documenti con `"`, `\`, a capo, tab, `<`, emoji, testo non latino | `tok_char` giusti |
| Per ogni frase gold, i token con etichetta 1 coprono la frase (a meno dei bordi) | tutte le righe |
| KnowLine letto due volte: `doc_tok` cade nella seconda copia | sempre |
| Senza preambolo, `input_ids` = piano binario | identici |
| P(supportato) dai logit salvati = verdetto scelto al passo 5 | differenza < 0,005 |

# Passo 7 — Metodi semplici da battere

## In breve

Prima di addestrare la testa si misura quanto ottengono metodi semplici, con le stesse metriche (10) e la stessa decodifica in tratti (08 §4). Script: `evidence/token_baselines.py`.

Ogni metodo dà un punteggio per token; poi stessa regola per tutti: scelta di una soglia su metà del dev (A), misura sull'altra metà (B).

## 1. I metodi

| Nome | Punteggio del token t | Costo |
|---|---|---|
| **E-RAND** | casuale, poi decodifica: tratti casuali | pavimento |
| **E-LEX** | 1 se la parola del token (minuscola, senza stopword) compare nel claim; poi media su una finestra di 15 token | nullo; riferimento principale |
| **E-EMB** | somiglianza fra l'embedding del claim e quello della finestra di 30 token centrata su t (modello piccolo, es. `BAAI/bge-small-en-v1.5`) | un encoder in più |
| **E-ATT** | attenzione della posizione di risposta verso t (`att[t]` salvata al passo 6), combinazione migliore di layer e teste scelta su A, media su una finestra di 9 token | nullo |
| **E-ABL** | gruppo di frasi trovato con l'ablazione (offline, frasi del nostro divisore), tutti i loro token a 1 | molti forward: solo riferimento |

E-LEX ed E-EMB non trovano le parti "ponte", che non condividono parole con il claim: sull'evidenza lontana ci si aspetta che crollino.

E-ATT è per natura una baseline per token: dice se il modello "guarda" già l'evidenza, e quanto serve addestrare una testa sopra.

## 2. Comandi

```
python -m evidence.token_baselines --method rand lex emb --block dev_evid_select
python -m evidence.token_baselines --method att --model knowline-4b --block dev_evid_select
gpu-run --batch --name evtok-eabl -- bash -c \
  'python -m evidence.token_baselines --method abl --model knowline-4b --block dev_evid_select --per-source 300 \
     > logs/evtok-eabl.out 2>&1'
```

## 3. Tabella da compilare (una per modello)

| Metodo | F1 caratteri HoVer | Frasi gold colpite HoVer | Insieme completo HoVer | F1 caratteri SciFact | F1 evidenza a ≥ 10 frasi | Tratti per riga |
|---|---|---|---|---|---|---|
| E-RAND | | | | | | |
| E-LEX | | | | | | |

| Metodo | F1 caratteri HoVer | Frasi gold colpite HoVer | Insieme completo HoVer | F1 caratteri SciFact | F1 evidenza a ≥ 10 frasi | Tratti per riga |
|---|---|---|---|---|---|---|
| E-EMB | | | | | | |
| E-ATT | | | | | | |
| E-ABL | | | | | | |

# Passo 8 — La testa, la loss, la decodifica

## In breve

Tre file:

- `evidence/token_head.py`: la testa. Riceve i vettori dei token del documento e restituisce un punteggio per token (`s`) e la soglia del documento (`s0`);
- `evidence/token_losses.py`: la loss, che dice alla testa quanto sbaglia;
- `evidence/decode.py`: trasforma i punteggi in tratti di testo (inizio e fine in caratteri). È la stessa funzione in valutazione e in produzione.

Ogni parte della testa si accende o spegne con un'opzione: gli esperimenti (11) usano lo stesso codice.

Il codice di questo file è stato eseguito su dati finti: forme, gradienti, righe di lunghezza diversa nello stesso batch, valori numerici e decodifica tornano.

## 1. La testa, passo per passo

1. **Vettore del token**: i vettori dei 2–4 layer (già ridotti a 256 dalla PCA, 01 §3) si sommano con pesi appresi; si aggiungono l'attenzione che la posizione di risposta dà al token (128 numeri) e il vettore `q` (uguale per tutti i token: dice alla testa qual è il claim e il verdetto). Tutto proiettato a 256.
2. **Contesto locale**: 3 convoluzioni 1D residue con dilatazione 1, 2, 4 e finestra 5: ogni token vede ~30 token intorno. Serve a capire che una parola è evidenza perché sta in una certa frase.
3. **Contesto globale**: i token si raggruppano in **blocchi fissi di 16** (per posizione, non per significato); ogni blocco è la media dei suoi token. Un transformer non causale a 2 layer lavora su `[q, blocco 1, …, blocco B]`: ogni blocco vede tutti gli altri e `q`. È ciò che permette di trovare evidenza sparsa (frase 1 e frase 40). Con 2.000 token sono 125 blocchi: costo trascurabile.
4. **Ritorno ai token**: ogni token riceve il contesto del suo blocco, lo combina con il proprio vettore e produce il suo punteggio `s[t]`. Dallo stato di `q` dopo il transformer esce la soglia `s0`.

```
import torch
import torch.nn as nn
import torch.nn.functional as F


class TokenEvidenceHead(nn.Module):
    def __init__(self, d_q, n_layers=2, d_in=256, n_att=128, d=256, block=16, kernel=5, conv_layers=3,
                 enc_layers=2, heads=4, max_blocks=4096, use_att=True, use_local=True, use_global=True):
        super().__init__()
        self.block, self.use_att, self.use_local, self.use_global = block, use_att, use_local, use_global
        self.mix = nn.Parameter(torch.zeros(n_layers))                # pesi dei layer (softmax)
        self.ln_h, self.ln_q = nn.LayerNorm(d_in), nn.LayerNorm(d_q)
        self.proj_h, self.proj_a, self.proj_q = nn.Linear(d_in, d), nn.Linear(n_att, d), nn.Linear(d_q, d)
        self.convs = nn.ModuleList([nn.Conv1d(d, d, kernel, padding=2 ** i * (kernel // 2), dilation=2 ** i)
                                    for i in range(conv_layers)])     # contesto locale: ~30 token
        self.conv_norms = nn.ModuleList([nn.LayerNorm(d) for _ in range(conv_layers)])
        self.pos, self.kind = nn.Embedding(max_blocks, d), nn.Embedding(2, d)
        enc = nn.TransformerEncoderLayer(d, heads, 4 * d, dropout=0.1, batch_first=True, norm_first=True)
        self.encoder = nn.TransformerEncoder(enc, enc_layers)         # contesto globale, sui blocchi
        self.back = nn.Linear(2 * d, d)
        self.out = nn.Linear(d, 1)

    def mixed(self, x, ln):                                           # x: [..., L, D]
```

```
        return ln((self.mix.softmax(0)[:, None] * x).sum(-2))

    def forward(self, h, att, q, mask):
        """h: [B, N, L, 256]; att: [B, N, 128]; q: [B, L, Dq]; mask: [B, N] (1 = token del documento)
        -> s: [B, N] punteggi dei token, s0: [B] soglia del documento."""
        m = mask.unsqueeze(-1).float()
        qv = self.proj_q(self.mixed(q, self.ln_q))                    # [B, d]
        x = self.proj_h(self.mixed(h, self.ln_h)) + qv[:, None]
        if self.use_att:
            x = x + self.proj_a(att.float())
        x = x * m
        if self.use_local:
            for conv, norm in zip(self.convs, self.conv_norms):       # convoluzioni residue, padding a zero
                x = x + F.gelu(conv(norm(x).transpose(1, 2)).transpose(1, 2)) * m
        B, N, d = x.shape
        nb = -(-N // self.block)
        xb = F.pad(x, (0, 0, 0, nb * self.block - N)).view(B, nb, self.block, d)
        mb = F.pad(m, (0, 0, 0, nb * self.block - N)).view(B, nb, self.block, 1)
        blocks = (xb * mb).sum(2) / mb.sum(2).clamp_min(1)              # media dei token di ogni blocco
        block_mask = mb.sum(2).squeeze(-1) > 0                           # [B, nb]
        x0 = qv + self.kind.weight[0]
        if self.use_global:
            seq = torch.cat([x0[:, None], blocks + self.pos(torch.arange(nb, device=x.device)) +
↪   self.kind.weight[1]], 1)
            pad = torch.cat([torch.zeros_like(block_mask[:, :1]), ~block_mask], 1)
            X = self.encoder(seq, src_key_padding_mask=pad)
            X0, Xb = X[:, 0], X[:, 1:]
        else:
            X0, Xb = x0, blocks
        ctx = Xb.repeat_interleave(self.block, dim=1)[:, :N]            # ogni token riceve il contesto del suo
↪   blocco
        z = F.gelu(self.back(torch.cat([x, ctx], -1)))
        s = self.out(z).squeeze(-1).masked_fill(mask == 0, -1e4)
        s0 = self.out(X0).squeeze(-1)
        return s, s0
```

| Parametro | Default | Note |
|---|---|---|
| `n_layers` | 2 | layer letti (23 e 31); H6 prova 4 |
| `d`, `block` | 256, 16 | H7 prova blocchi da 8 e 32 |
| `conv_layers`, `kernel` | 3, 5 | finestra locale ~30 token |
| `enc_layers`, `heads` | 2, 4 | |
| Parametri | ~4,5 M (con `d_q` = 2.560) | |

**Configurazione di partenza H0**: un solo layer, niente attenzione, niente contesto locale né globale (`use_att=False, use_local=False, use_global=False`): un punteggio per token dal suo vettore e da `q`. Gli esperimenti aggiungono un pezzo alla volta.

## 2. Il problema delle etichette

Le fonti annotano **frasi**, non parole: in una frase gold tutti i token hanno etichetta 1, anche quelli irrilevanti (“is”, “the”, la data accessoria). Chiedere che ogni token gold superi la soglia insegnerebbe alla testa a evidenziare frasi intere sempre, e penalizzerebbe una testa che evidenzia solo la parte utile.

Per questo la loss ha più termini: uno per token, con peso moderato, e due che parlano di **frasi gold** e di **ciò che sta fuori**, che sono le cose che contano per l’utente:

- ogni frase gold deve essere **colpita**: almeno un suo token chiaramente sopra soglia;
- **fuori** dalle frasi gold nessun token deve superare la soglia (con peso ω, perché le annotazioni possono essere incomplete);
- i punteggi di token vicini devono essere simili (tratti contigui, non a macchia di leopardo).

## 3. La loss

Con `d[t] = s[t] − s0` (positivo = sopra soglia), per una riga:

```
supervised:  L = L_tok + 1,0 · L_hit + 0,5 · ω · L_out + 0,1 · L_tv
  L_tok = BCE pesata per token fra σ(d[t]) e l'etichetta (peso α = 3 sui token gold, ω sugli altri)
  L_hit = media sulle frasi gold g di softplus(1 − max_{t ∈ g} d[t])         ogni frase gold colpita
  L_out = softplus(max_{t ∉ gold} d[t] + 1)                                 niente acceso fuori
  L_tv  = media |σ(d[t+1]) − σ(d[t])|                                        tratti contigui
mil_neg:     L = 0,3 · softplus(max_t d[t] + 1)                             claim non supportato: nessun token
↪  sopra soglia
none:        L = 0
```

I documenti hanno lunghezze molto diverse: la loss si calcola **riga per riga** e poi si fa la media pesata con il peso della riga (`row_weight`, 04 §3).

```python
import torch
import torch.nn.functional as F


def token_evidence_loss(s, s0, y, mask, gold_spans, term, omega, alpha=3.0, margin=1.0,
                        b_hit=1.0, b_out=0.5, b_tv=0.1, mu_neg=0.3):
    """Una riga alla volta (i documenti hanno lunghezze diverse).
    s: [N] punteggi dei token; s0: [] soglia; y: [N] etichette 0/1; mask: [N]; gold_spans: [(a, b)] token,
↪   estremi inclusi;
    term: "supervised" | "mil_neg" | "none"; omega: peso dei token fuori dalle frasi gold."""
    d = (s - s0)[mask.bool()]
    y = y[mask.bool()].float()
    if term == "none":
        return s.sum() * 0.0
    if term == "mil_neg":                                          # nessun token sopra la soglia
        return mu_neg * F.softplus(d.max() + margin)
    w = torch.where(y > 0, torch.full_like(d, alpha), torch.full_like(d, omega))
    l_tok = (F.binary_cross_entropy_with_logits(d, y, reduction="none") * w).sum() / w.sum()
    l_hit = torch.stack([F.softplus(margin - d[a:b + 1].max()) for a, b in gold_spans]).mean()  # ogni frase gold
↪   colpita
    outside = d[y == 0]
    l_out = F.softplus(outside.max() + margin) if outside.numel() else d.sum() * 0.0      # niente acceso fuori
    p = torch.sigmoid(d)
    l_tv = (p[1:] - p[:-1]).abs().mean() if p.numel() > 1 else d.sum() * 0.0            # tratti contigui
    return l_tok + b_hit * l_hit + b_out * omega * l_out + b_tv * l_tv
```

Esempio numerico (anche come test automatico): 10 token, frase gold sui token 2–5, `s0 = 0`, ω = 0,5, punteggi `−3; −3; 2; 0,5; −0,5; 2; −3; −3; −3; 1,5` (il token 9, fuori dalla frase gold, è acceso): loss **1,4015**. La stessa riga come negativo (`mil_neg`): **0,9146** (il token più alto, 2, sta sopra soglia).

## 4. La decodifica: dai punteggi ai tratti

1. token con `s[t] − s0 > δ` (δ = 0 di default, scelto sul dev, 09 §2);
2. tratti di token consecutivi; si uniscono tratti separati da al massimo 3 token;
3. si tolgono i tratti sotto 4 token (parole isolate);
4. si tengono i 5 tratti con punteggio medio più alto, in ordine di posizione;
5. se il verdetto è “supportato” e non resta nulla, si mostra il tratto intorno al token migliore;
6. ogni tratto diventa `[inizio, fine)` di caratteri con `tok_char` (06 §1), allargato alla parola intera.

```python
import math


def decode(s, s0, tok_char, text, delta=0.0, max_gap=3, min_len=4, max_spans=5, force_one=True):
    """s: punteggi dei token del documento (lista); s0: soglia; tok_char: [(inizio, fine)] in caratteri del
    ↪  documento.
    -> lista di {"start", "end", "score"} sul testo originale, in ordine di posizione."""
    on = [x - s0 > delta for x in s]
    runs, i, n = [], 0, len(s)
    while i < n:                                                   # tratti consecutivi sopra soglia
        if on[i]:
            j = i
            while j + 1 < n and on[j + 1]:
                j += 1
            runs.append([i, j]); i = j + 1
        else:
            i += 1
    merged = []
    for a, b in runs:                                              # si chiudono i buchi piccoli
```

```
        if merged and a - merged[-1][1] - 1 <= max_gap:
            merged[-1][1] = b
        else:
            merged.append([a, b])
    merged = [r for r in merged if r[1] - r[0] + 1 >= min_len]
    if not merged and force_one and n:                      # supportato: almeno un tratto, intorno al token
    ↪  migliore
        k = max(range(n), key=lambda t: s[t])
        merged = [[max(0, k - min_len), min(n - 1, k + min_len)]]
    score = lambda r: sum(s[r[0]:r[1] + 1]) / (r[1] - r[0] + 1) - s0
    merged = sorted(sorted(merged, key=score, reverse=True)[:max_spans])
    out = []
    for a, b in merged:
        start, end = tok_char[a][0], tok_char[b][1]
        while start > 0 and not text[start - 1].isspace():  # si allarga alla parola intera
            start -= 1
        while end < len(text) and text[end].isalnum():
            end += 1
        while start < end and text[start].isspace():
            start += 1
        out.append({"start": start, "end": end, "score": 1 / (1 + math.exp(-score([a, b])))})
    return out
```

Esempio: testo “Rose Byrne is an Australian actress. She starred in Get Him to the Greek in 2010.”, token 0–6 sopra soglia tranne il 3 (un buco di un token), token 12 sopra soglia da solo → un tratto: “Rose Byrne is an Australian actress.” (il buco è chiuso, il token isolato è tolto).

Parametri della decodifica (δ, buco massimo, lunghezza minima, numero massimo di tratti) si scelgono sul dev e si scrivono in `decision.json` (12).

### 5. Test

`tests/test_token_head.py`:

- forme e maschere con righe di lunghezza diversa nello stesso batch; i token di riempimento hanno punteggio −10.000 e non influenzano gli altri;
- la loss dell’esempio numerico (tolleranza 1e-3); `none` dà zero e nessun gradiente;
- la decodifica dell’esempio; un documento senza token sopra soglia e `force_one` produce un tratto; i tratti non escono dal testo.

## Passo 9 — Training

### In breve

Come nel piano per frase, tre modi dal più semplice al più costoso; si passa al successivo solo se il precedente non basta.

| Fase | Il modello | Ingresso della testa | Quando |
|---|---|---|---|
| **1a** | congelato, non in memoria | vettori ridotti con la PCA, salvati al passo 6 | quasi tutti gli esperimenti |
| **1b** | congelato, in memoria | vettori completi ricalcolati a ogni passo; la testa impara la sua proiezione al posto della PCA | conferma (H9): la PCA perde qualcosa? |
| **2** | LoRA addestrata con la testa | come 1b, con gradiente | solo se gli obiettivi non sono raggiunti |

Nelle fasi 1a e 1b il verdetto non cambia. Ogni prova scrive in `data/evidence_tok/runs/<modello>/<id>/` configurazione, pesi della testa, log, metriche dev.

## 1. Fase 1a

| Impostazione | Valore |
|---|---|
| Ottimizzatore | AdamW, lr 5e-4, warmup 5%, coseno, weight decay 0,01 |
| Epoche | fino a 10; ci si ferma se per 2 epoche `dev_evid_select` non migliora |
| Batch | 16 righe, raggruppate per lunghezza (si riempiono fino alla più lunga del batch) |
| Dropout | 0,1 nel transformer; 0,1 sulle feature di attenzione (intere, per token) |
| Checkpoint tenuto | media più alta di F1 sui caratteri e frasi gold colpite, su `dev_evid_select` |
| Loss | 08 §3, media sulle righe pesata con `row_weight` |

```
python -m evidence.token_train --phase 1a --model knowline-4b --features data/evidence_tok/features/knowline-4b \
    --mix ev_tok_head --config configs/tok_h0.json --run h0
```

Prima della prima prova: la testa deve imparare a memoria 64 righe (frasi gold colpite ≈ 1, niente acceso fuori). Se non ci riesce, c'è un errore in `tok_char`, nelle etichette o nel codice.

Memoria: per una riga di 6.000 token la testa usa poche decine di MB (il transformer lavora su ~375 blocchi). La fase 1a gira anche su CPU, più comodamente su una GPU qualsiasi.

## 2. Scegliere i parametri della decodifica

Dopo il training, su `dev_evid_select`, fra le righe supportate che il verdetto giudica supportate, griglia su:

| Parametro | Valori |
|---|---|
| δ (scarto dalla soglia) | da −1 a 1, passo 0,1 |
| buco massimo | 0, 2, 3, 5 token |
| lunghezza minima | 2, 4, 6 token |
| tratti massimi | 3, 5, 8 |

Criterio: media di F1 sui caratteri e frasi gold colpite. Si conferma su `dev_evid_confirm` (non più di 1 punto sotto). I valori vanno in `decision.json`.

## 3. Che cosa guardare durante il training

| Cosa | Su | Segnale di problema | Cosa fare |
|---|---|---|---|
| F1 sui caratteri, frasi gold colpite | `dev_evid_select` | non supera E-LEX dopo 2 epoche | controllare `tok_char` ed etichette prima di toccare la testa |
| Tratti per riga, lunghezza media dei tratti | `dev_evid_select` | tratti troppo lunghi (paragrafi interi) o troppi frammenti | peso di `L_out`, `L_tv`; parametri della decodifica |
| Frasi gold colpite per distanza fra le frasi gold | righe sparse | crollo oltre 10 frasi | contesto globale (H3), più righe sparse |
| Righe con tratti sopra soglia | `dev_evid_neg` | oltre il 30% | peso di `mil_neg` più alto |
| Pesi dei layer | testa | tutto su un layer | basta quel layer (meno spazio su disco) |

### 4. Fase 1b: vettori completi, proiezione appresa

Il modello è in memoria, congelato. A ogni passo si rifà il forward, si prendono i vettori **completi** dei token del documento (2.560 o 4.096 numeri) e la testa li proietta con una matrice appresa (`nn.Linear(D, 256)`, inizializzata con la PCA).

| Impostazione | Gruppo 4B | Gruppo 9B |
|---|---|---|
| GPU | 24 GB | 45 GB |
| Righe per passo | 2–4 (accumulando fino a 16) | 1 (accumulando fino a 16) |
| Lunghezza massima | 8.192 token | 8.192 token |
| Partenza | la testa migliore della fase 1a | idem |
| Epoche | 2 | 1 |

### 5. Fase 2: training anche del modello

Come nel piano per frase (../testa_per_frase/09_training.md §5): partenza dalla LoRA del verdetto del piano binario e dalla testa della fase 1; lr della LoRA 5e-6 – 1e-5; loss = loss del verdetto + 0,3 × loss di evidenza; dati del piano binario più le righe con evidenza. Il verdetto si controlla a ogni checkpoint: si accetta solo entro 0,3 punti di BAcc e 0,01 di ECE dal piano binario.

### 6. Test prima di ogni nuova configurazione

| Test | Atteso |
|---|---|
| Imparare a memoria 64 righe | frasi gold colpite ≈ 1, F1 sui caratteri > 0,9 |
| H0 riprodotto con le opzioni spente | stessi numeri (stesso seme) |
| Righe `none` | loss e gradiente zero |
| Batch con righe di lunghezze molto diverse | stessi punteggi di una riga alla volta (a meno di 1e-4) |

## Passo 10 — Valutazione

### In breve

Per ogni modello si misura che il verdetto non sia cambiato, che i tratti mostrati siano quelli giusti e che la latenza resti quella di un decision model. Script: `evidence/token_eval.py`, `evidence/latency.py`.

Le metriche si calcolano sulle righe supportate **che il modello giudica supportate** (solo lì si mostrano tratti). A parte si riporta quante righe supportate restano senza evidenza perché il verdetto dice “no”.

### 1. Metriche sui caratteri (indipendenti da qualunque divisione in frasi)

Si confrontano i caratteri dei tratti mostrati con quelli delle frasi gold (`gold_char_sets`), spazi esclusi:

| Metrica | Definizione |
|---|---|
| Precisione | quota dei caratteri mostrati che sta dentro una frase gold |
| Recall | quota dei caratteri delle frasi gold che è mostrata |
| F1 sui caratteri | media armonica delle due, per riga, poi media sulle righe |
| AUPRC per token | dai punteggi `s[t] − s0`, prima della decodifica: quanto bene i punteggi ordinano i token |

Con più insiemi alternativi (WiCE) si prende, per riga, l'insieme che dà l'F1 più alta.

Attenzione: la recall sui caratteri **penalizza** una testa che evidenzia solo la parte utile di una frase gold (le etichette sono frasi intere). Per questo non è la metrica principale: lo sono le due del §2.

## 2. Metriche sulle frasi gold (quelle che contano)

| Metrica | Definizione |
|---|---|
| **Frasi gold colpite** | quota di frasi gold con almeno metà dei caratteri, **oppure** almeno 5 token, dentro i tratti mostrati |
| **Insieme completo** | quota di righe in cui tutte le frasi gold (di almeno un insieme) sono colpite |
| Testo fuori | quota dei caratteri mostrati fuori da ogni frase gold (è 1 − precisione) |
| Frasi gold colpite per distanza | fasce: vicine / 2–9 / 10–19 / ≥ 20 frasi fra la prima e l'ultima frase gold |
| Frasi gold colpite per distanza dall'inizio del documento | < 1.024 / 1.024–4.096 / > 4.096 token |
| Tratti per riga, token per tratto | media e 95° percentile |
| **Sufficienza** | il documento ridotto ai soli tratti mostrati fa ancora dire "supportato" al modello |
| Necessità | il documento senza i tratti mostrati fa dire "non supportato" (si riporta, non è un obiettivo) |
| Negativi accesi | su `dev_evid_neg`: righe con almeno un tratto sopra soglia |

Sufficienza e necessità costano 2 forward per riga: solo in valutazione. Per la sufficienza i tratti si uniscono con " … " fra uno e l'altro.

## 3. Confronto con la testa per frase

Per confrontare le due teste sullo stesso terreno, i tratti si convertono in frasi (solo per la valutazione): una frase della riga (`sentences`, il nostro divisore) è "mostrata" se almeno metà dei suoi caratteri è dentro un tratto. Poi si calcolano le metriche per frase del piano per frase (../testa_per_frase/10_valutazione.md §2). Il divisore qui serve solo a misurare: non entra nel modello.

## 4. Prove di comportamento

| Prova | Atteso |
|---|---|
| Evidenza spostata all'inizio / a metà / alla fine | stesse frasi gold colpite in ≥ 90% delle righe |
| Evidenza oltre 1.024 e oltre 4.096 token | idem |
| Frasi di disturbo aggiunte | nessun tratto dentro il disturbo in ≥ 95% |
| Stesso documento, punteggiatura tolta (frasi senza punti) | frasi gold colpite al massimo 5 punti sotto: è il caso in cui la testa per frase non funziona |
| Frase gold riscritta senza parole in comune col claim | colpita in ≥ 80% |
| Coppia quasi identica (positivo / negativo con un fatto tolto) | positivo: tratti giusti; negativo: nessun tratto sopra soglia |

## 5. Latenza

Come nel piano per frase: stesso server, 1.000 richieste reali; A = solo verdetto senza preambolo, B = solo verdetto con preambolo, C = verdetto + tratti. Obiettivo: C ≤ B + 3% (modelli 4B: +3% o +3 ms).

In più rispetto al piano per frase: la proiezione dei vettori di tutti i token (n × D × 256, sulla GPU) e le convoluzioni. Si misurano a parte. Ordine di grandezza per 6.000 token su un 4B: qualche GFLOP, cioè frazioni di millisecondo su GPU.

### 6. Comandi

```
python -m evidence.token_eval --model knowline-4b --run data/evidence_tok/runs/knowline-4b/<id> --block
↪  dev_evid_select
# test: una volta sola, a configurazione congelata
gpu-run --batch --name evtok-test -- bash -c \
  'python -m evidence.token_eval --model knowline-4b --release <rilascio> \
     --block test_evid_id test_evid_ood test_evid_sparse --sufficiency > logs/evtok-test.out 2>&1'
```

Lo script salva `metrics.json` e `predictions.jsonl` (per riga: tratti, punteggi per token compressi, soglia), così le metriche si ricalcolano senza rifare i forward.

### 7. Report finale (uno per modello)

| Set | Metrica | E-LEX | E-ATT | E-ABL | Testa per frase | Testa per token |
|---|---|---|---|---|---|---|
| test_evid_id | frasi gold colpite / insieme completo / F1 caratteri | | | | | |
| test_evid_ood | idem | | | | | |
| test_evid_sparse | frasi colpite a ≥ 20 frasi di distanza | | | | | |
| prova senza punteggiatura | frasi gold colpite | | | | | |
| tutti dev_evid_neg | sufficienza negativi accesi | | | | | |
| verdetto | BAcc (uguale al piano binario?) | — | — | — | | |
| latenza | p50 / p95, C contro B | | | | | |

# Passo 11 — Esperimenti

### In breve

Si parte da una testa minima (H0) e si aggiunge **un pezzo alla volta**, tenendolo solo se migliora. Tutto su **KnowLine-4B** con i vettori salvati (fase 1a). Alla fine elephant-1 e gli altri modelli ripetono la configurazione migliore.

### Regole

1. Una modifica per prova, con il suo `configs/tok_<id>.json`.

2. Si decide su `dev_evid_select` con: frasi gold colpite, insieme completo, F1 sui caratteri, frasi colpite con evidenza a ≥ 10 frasi di distanza; più i negativi accesi su `dev_evid_neg`.
3. Una modifica si tiene se migliora le frasi colpite di ≥ 1 punto o l'insieme completo di ≥ 2, senza peggiorare le altre metriche di più di 1 punto (e restando nel limite di latenza).
4. A parità, la versione più semplice.
5. Guadagni sotto 2 punti si ripetono con 2 semi in più.

## La testa di partenza: H0

Un layer (31), niente attenzione, niente contesto locale né globale: punteggio per token dal suo vettore più `q`. Loss completa (08 §3), decodifica di default.

Prima domanda: H0 batte E-LEX ed E-ATT? Se no, si controllano `tok_char` ed etichette (06 §7).

## Esperimenti principali (in quest'ordine)

| Nome | Che cosa si aggiunge | Che cosa si guarda |
|---|---|---|
| H1 | contesto locale (3 convoluzioni) | frammentazione dei tratti, F1 sui caratteri |
| H2 | attenzione della posizione di risposta per token | frasi colpite, soprattutto le "ponte" |
| H3 | contesto globale (transformer sui blocchi) | evidenza lontana, insieme completo |
| H4 | soglia del documento `s0` contro soglia fissa sul dev | insieme completo; numero di tratti |
| H5 | termini della loss: senza `L_hit`; senza `L_out`; senza `L_tv` (uno alla volta) | quanto serve ciascuno |
| H6 | 4 layer (19, 23, 27, 31) invece di 2 | frasi colpite; spazio su disco |
| H7 | blocchi da 8 e da 32 token invece di 16 | evidenza lontana, latenza |

## Controlli sulla configurazione migliore

| Nome | Che cosa si cambia | Che cosa dice |
|---|---|---|
| H8 | vettori estratti senza preambolo | quanto vale il preambolo (anche per un modello bidirezionale: elephant-1) |
| H9 | fase 1b: vettori completi e proiezione appresa invece della PCA | se la PCA perde informazione |
| H10 | training con le righe C2D a intervalli esatti (03 §1) | se la testa impara tratti più stretti di una frase |
| H11 | ω: 0,3 / 0,5 / 1 | gestione delle annotazioni incomplete |
| H12 | peso delle righe HoVer `db_split`: 0 / 0,5 / 1 | se le righe recuperate dal db aiutano o fanno rumore |
| H13 | decodifica: con e senza allargamento dei tratti alla frase (divisore qualunque, solo in visualizzazione) | quanto cambia la leggibilità rispetto alle metriche |

### Confronto con la testa per frase

| Nome | Che cosa |
|---|---|
| T-vs-F | stessa fase (1a), stessi dati e stesso modello: testa per token contro testa per frase (../testa_per_frase/), con le metriche per frase (10 §3) e con la prova senza punteggiatura |

Lettura: se la testa per token è entro ~1 punto della testa per frase sulle metriche per frase, e molto meglio senza punteggiatura, si adotta la testa per token (nessun divisore in produzione). Se è nettamente sotto, il costo dell'indipendenza dal divisore è troppo alto e si torna alla testa per frase, magari con un divisore incluso nella repo.

### elephant-1 e gli altri modelli

| Nome | Che cosa |
|---|---|
| E0 | elephant-1: PCA, estrazione, configurazione migliore di KnowLine |
| X0 | gli altri decision model, nell'ordine del piano binario |
| E2 | fase 2, solo se su KnowLine ha dato un guadagno chiaro |
| L1 | latenza per ogni modello rilasciato |

### Controlli finali

| Nome | Che cosa |
|---|---|
| C1 | 3 semi della testa finale: media e variabilità |
| C2 | sufficienza e necessità su tutti i set di test |
| C3 | testa contro E-ABL |
| C4 | confronto fra i modelli sulle righe giudicate supportate da tutti |

### Registro

`data/evidence_tok/runs/registry.md`, una riga per prova:

| Nome | Modello | Fase | Modifica | Frasi colpite | Insieme completo | F1 caratteri | Colpite ≥ 10 frasi | Tratti per riga | Negativi accesi | Sufficienza | Δ latenza | Decisione |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| E-LEX | — | — | — | | | | | | | | — | riferimento |
| H0 | KnowLine-4B | 1a | — | | | | | | | | | |

# Passo 12 — Server e rilascio

### In breve

Un server (`evidence/serve_tokens.py`) che per ogni richiesta fa **un forward** e restituisce verdetto e tratti di evidenza. **Nessun divisore in frasi**: il documento entra così com'è, l'uscita sono intervalli di caratteri. Usa lo stesso codice di rendering, estrazione e decodifica della valutazione.

## 1. Che cosa fa il server per ogni richiesta

```
richiesta {document, claim, [display: "spans" | "sentences"]}
  1. prompt con il preambolo (codice degli autori) → input_ids; token del documento e loro caratteri (tok_char)
  2. un forward con gli hook: punteggi delle opzioni, vettori dei token del documento (proiettati), attenzione
  3. P(supportato); se non supera la soglia → risposta senza evidenza
  4. testa → s[t], s0 → decodifica → tratti in caratteri sul documento originale
  5. (solo se display = "sentences") ogni tratto si allarga alla frase che lo contiene, con il divisore del server
```

```python
def answer(document, claim, display="spans"):
    ex = render_tokens(document, claim, preamble=DECISION["preamble"])          # input_ids, doc_tok, tok_char
    logits, feats = model.forward(ex)
    p = p_supported(logits, DECISION["temperature"])
    if p <= DECISION["threshold"]:
        return {"p_supported": p, "supported": False, "evidence": []}
    s, s0 = head(**feats)
    spans = decode(s.tolist(), float(s0), ex["tok_char"], document, **DECISION["decode"])
    if display == "sentences":
        spans = expand_to_sentences(spans, document)                            # solo visualizzazione
    return {"p_supported": p, "supported": True, "evidence": spans}
```

Il passo 5 è l'unico punto in cui un divisore può comparire, e solo per mostrare. Cambiarlo (NLTK, regex, quello del client) non tocca la testa né i punteggi.

## 2. Come gira ogni modello

Come nel piano per frase (../testa_per_frase/12_export_e_servizio.md §2): transformers con il codice degli autori e gli hook. Per OneJev, Nimble e StartLux il forward è in transformers con il prompt degli autori (i loro runtime non espongono gli stati), con il verdetto verificato contro il loro server.

Regole:

- la proiezione dei vettori dei token a 256 si fa sulla GPU nell'hook: escono n × 256 numeri, non n × D;
- per KnowLine, nelle richieste lette due volte, si usano i token della seconda copia;
- documenti oltre il limite di token: errore esplicito (`document_too_long`), mai troncati.

## 3. La cartella di rilascio

`releases/<modello>-fc-evtok-<versione>/`:

| File | Contenuto |
|---|---|
| pesi del verdetto | quelli del piano binario (o riaddestrati col preambolo, o con la fase 2) |
| `token_head.safetensors` | pesi della testa (~18 MB) |
| `projection/` | matrici di proiezione per layer (PCA, o la proiezione appresa della fase 1b) |
| `decision.json` | impostazioni (sotto) |
| `frozen.json` | hash di tutto: pesi, testa, proiezioni, `token_render.py`, `escape.py`, `decode.py`, `question.json`, split, mix |
| `report.md`, `MODEL_CARD.md` | risultati e descrizione |

```json
{"model": "knowline-4b-fc-evtok-v1",
 "question_id": "…", "take": "…",
 "preamble": "Claim under review: {claim}",
 "temperature": 1.0, "threshold": 0.50,
 "evidence": {"layers": [23, 31], "attention_layers": [3, 7, 11, 15, 19, 23, 27, 31], "block": 16,
              "head_config": "configs/tok_<id>.json"},
 "decode": {"delta": 0.0, "max_gap": 3, "min_len": 4, "max_spans": 5, "force_one": true},
 "max_prompt_tokens": 32768}
```

(Valori di esempio.) Non c'è nessun campo per il divisore: il modello non ne ha bisogno.

### 4. Controlli prima del rilascio

| Controllo | Deve dare |
|---|---|
| Verdetto del server = verdetto in valutazione (200 righe) | differenza media di P < 0,005 |
| Tratti del server = tratti in valutazione (200 righe) | identici in almeno 199 righe |
| Richieste in batch con lunghezze diverse | stessi tratti che una alla volta |
| Documenti con ", \, a capo, <, emoji, testo non latino | `document[start:end]` è il testo atteso |
| Documento senza punteggiatura | risposta valida, tratti dentro il testo |
| Latenza | entro il limite (10 §5) |

### 5. Congelamento e test

`frozen.json` prima dei test; test eseguiti una sola volta; `report.md`. Model card: verifica documento–claim in inglese; evidenzia **solo** quando il claim è supportato; restituisce tratti di testo, non frasi; i tratti sono stati addestrati da annotazioni a livello di frase, quindi tendono a coprire frasi intere o loro parti significative. Licenze: quelle dei pesi di partenza (StartLux e Nimble CC BY-NC).

## Checklist e problemi frequenti

### 1. Checklist per passo

**Passo 1 — Modelli e hook per token**

- ☐ Hook sui token del documento con proiezione nell'hook; test del piano per frase superati (verdetto, layer, attenzione, maschera).
- ☐ PCA per KnowLine-4B (layer 23 e 31) con la varianza spiegata registrata.

**Passo 2 — Dati**

- ☐ `gold_char_sets` e `contra_char` aggiunti a `evidence/data/schema.py`, validati e testati; dataset ricostruiti.
- ☐ 30 positivi per fonte letti con gli intervalli evidenziati.

**Passo 3 — Dati generati** (facoltativo per le prime prove)

- ☐ C2D con intervalli esatti; ablazione con uscita in intervalli; righe sparse.

**Passo 4 — Split e mix**

- ☐ Split per `doc_id`; WiCE con `aggrefact_overlap` fuori dal training; dev e test di HoVer solo `exact`.
- ☐ Mix `ev_tok_head` con i vincoli rispettati.

**Passo 5 — Preambolo**

- ☐ Scelto come nel piano per frase; `question.json` aggiornato.

**Passo 6 — Rendering ed estrazione**

- ☐ `escape_with_map` spostata in `evidence/escape.py`; mappa inversa testata.
- ☐ `tok_char` ricostruisce il documento (200 su 200); etichette dei token coprono le frasi gold.
- ☐ Vettori di KnowLine-4B salvati per il mix e i set dev.

**Passo 7 — Baseline**

- ☐ E-RAND, E-LEX, E-EMB, E-ATT; E-ABL su 300 righe per fonte.

**Passo 8 — Testa, loss, decodifica**

- ☐ `tests/test_token_head.py` passa (forme, esempio numerico, decodifica).

**Passo 9 — Training**

☐ Impara a memoria 64 righe; H0 riproducibile; parametri della decodifica scelti e confermati.

**Passo 10 — Valutazione**

☐ Metriche sui caratteri e sulle frasi gold; confronto con la testa per frase; prova senza punteggiatura; latenza.

**Passo 11 — Esperimenti**

☐ H0–H7, controlli H8–H13, T-vs-F; registro compilato.

**Passo 12 — Rilascio**

☐ Controlli del server superati; `frozen.json` prima dei test; model card.

### 2. Se succede…

| Problema | Prima cosa da controllare | Poi |
|---|---|---|
| La testa non impara (F1 vicino al caso) | `tok_char` e etichette dei token (passo 6, test) | `meta.json` dei vettori contro la configurazione |
| Evidenzia paragrafi interi | peso di `L_out`; decodifica (buco massimo troppo grande) | ω più alto; lunghezza massima dei tratti |
| Tratti frammentati, parole isolate | `L_tv`; contesto locale (H1) | lunghezza minima della decodifica |
| Colpisce una frase gold ma non la “ponte” | contesto globale (H3), attenzione (H2) | più righe sparse; blocchi più piccoli (H7) |
| Fa come E-LEX (accende parole del claim) | negativi nel mix e loro peso | più negativi con parole in comune col claim |
| Accende tratti sui claim non supportati | peso di `mil_neg` | soglia δ più alta |
| Precisione bassa ma sufficienza alta | leggere a mano i “falsi positivi”: spesso evidenza valida non annotata | ω più basso per quella fonte |
| Peggio della testa per frase sulle metriche per frase | rumore delle etichette per token | righe C2D a intervalli esatti (H10); `L_hit` più forte |
| Spazio su disco insufficiente | numero di layer, componenti PCA | 1 layer; 128 componenti |
| Latenza oltre il limite | tempo di proiezione, convoluzioni, transformer misurati a parte | blocchi più grandi; meno convoluzioni |
| Tratti con caratteri sbagliati nel testo originale | mappa inversa dell’escaping | test con documenti con `"`, `\`, a capo |

## Una strada da esplorare: etichette per token dal modello stesso

### Il problema

Le etichette della testa per token sono grossolane: le fonti segnano frasi intere, quindi dentro una frase gold anche le parole irrilevanti hanno etichetta 1. La loss compensa (chiede di colpire ogni frase gold, non di accenderla tutta), ma la testa non riceve mai un segnale su **quale parte** della frase conta. Il vantaggio principale della testa per token, cioè evidenziare solo ciò che serve, resta quindi poco sfruttato.

### L’idea

Ricavare etichette più fini **dal decision model stesso**, offline, togliendo pezzi più piccoli di una frase e guardando quanto cambia il verdetto:

```
per ogni frase gold g di una riga positiva:
    si divide g in pezzi di ~5 token (finestre fisse, nessun divisore)
    per ogni pezzo w:  importanza(w) = z(documento) − z(documento senza w)        z = margine del verdetto
```

I pezzi con importanza alta sono la parte della frase che regge il verdetto (il nome, la data, la relazione); quelli con importanza vicina a zero sono contorno. Le etichette della testa diventano morbide: 1 sui pezzi importanti, un valore più basso (es. 0,3) sugli altri token della frase gold, 0 fuori.

## Come farlo

1. **Righe**: i positivi del training con frasi gold (HoVer `exact`, SciFact, WiCE train).
2. **Calcolo** (`evidence/label_token_importance.py`), con KnowLine-4B: un forward per pezzo, solo dentro le frasi gold (in media 3 frasi × ~6 pezzi = ~18 forward per riga). KnowLine è causale, quindi i prompt sono uguali fino al pezzo tolto e si riusa il calcolo della parte iniziale comune.
3. **Prima verifica, senza training**: su 50 righe lette a mano, i pezzi con importanza più alta sono davvero la parte che dimostra il claim? Se no (il modello segue scorciatoie, o l'importanza è piatta), ci si ferma.
4. **Training** (esperimento H-IMP, fase 1a): `L_tok` usa le etichette morbide al posto di quelle per frase; `L_hit` e `L_out` restano uguali.
5. **Valutazione**: le metriche di 10 più una nuova, la **lunghezza media dei tratti a parità di frasi gold colpite** (più corta = più precisa), e la sufficienza (i tratti più corti reggono ancora il verdetto?).

## Quando tenerlo

Rispetto alla configurazione migliore senza H-IMP:

- frasi gold colpite e insieme completo non più di 1 punto sotto;
- tratti almeno del 30% più corti;
- sufficienza non più di 2 punti sotto.

## Rischi

| Rischio | Cosa fare |
|---|---|
| Togliere 5 token rende la frase sgrammaticata e il modello reagisce alla forma, non al contenuto | confrontare con il togliere le stesse posizioni sostituendole con "…"; tenere l'importanza solo dove le due misure concordano |
| Informazione ripetuta nella frase: importanza bassa per tutti i pezzi | le etichette morbide non scendono mai sotto 0,3 dentro la frase gold |
| Costo: ~18 forward per riga | solo frasi gold, riuso della parte comune del prompt, KnowLine-4B |

## Perché vale la pena

È il modo più diretto di usare ciò che la testa per token sa fare e quella per frase no: mostrare la parte della frase che conta. Non richiede nuove annotazioni umane, è tutto offline e in produzione non cambia nulla.